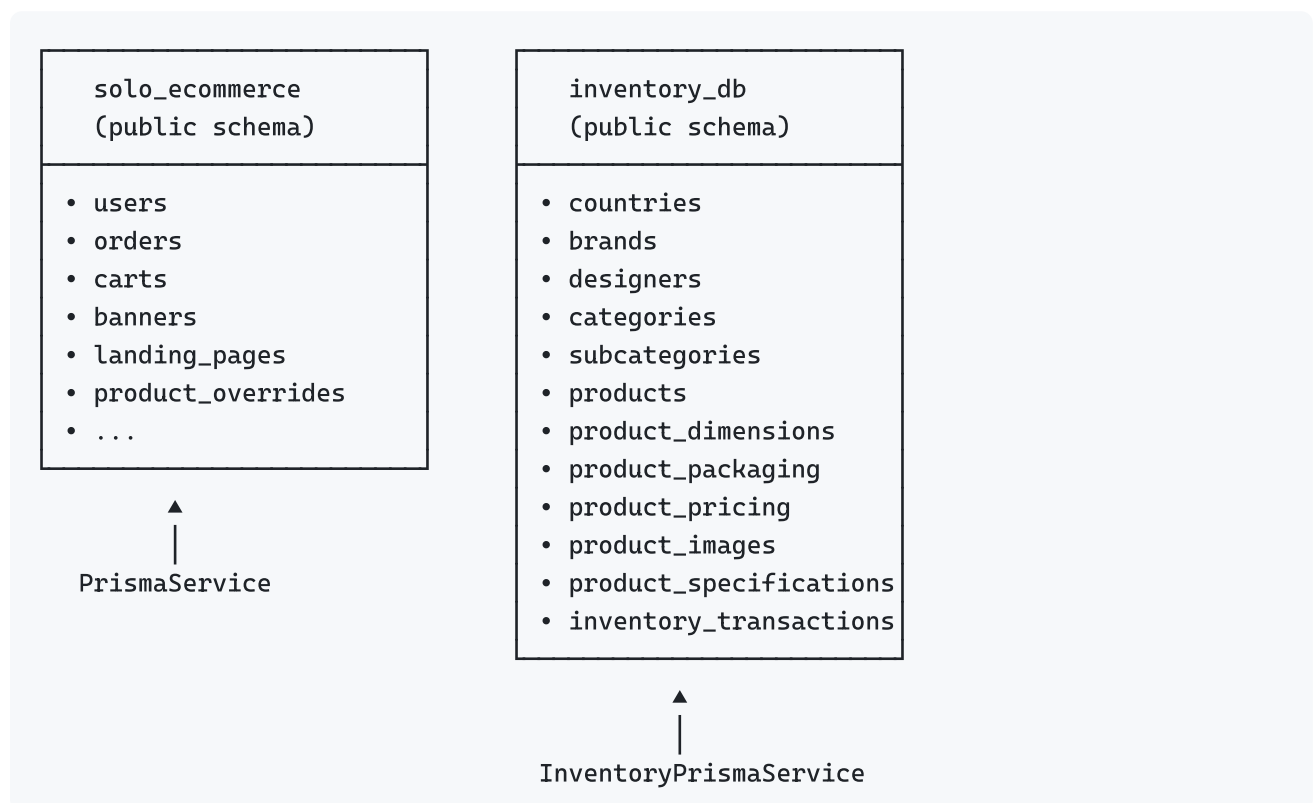


Single Database + Inventory Schema Runbook

Overview

This runbook documents the migration from a two-database architecture to a single database (`solo_ecommerce`) with a dedicated `inventory` schema for catalog data.

Before Migration



After Migration

solo_ecommerce	
public schema	inventory schema
• users • orders • carts • banners • landing_pages • product_overrides • departments • categories (public) • brands (public) • products (public) • ...	• countries • brands • designers • categories • subcategories • products • product_dimensions • product_packaging • product_pricing • product_images • product_specifications • inventory_transactions


 PrismaService (single client)

Prerequisites

1. **PostgreSQL 12+** running on localhost:5432
2. **Node.js 18+**
3. **Backup both databases** before starting

```
# Backup commands
pg_dump -U postgres -h localhost solo_ecommerce > solo_ecommerce_backup.sql
pg_dump -U postgres -h localhost inventory_db > inventory_db_backup.sql
```

Migration Steps

Step 1: Run the Migration Script

The migration script uses `postgres_fdw` to copy data from `inventory_db` into a new `inventory` schema within `solo_ecommerce`.

```
cd backend
npx ts-node scripts/migrate_inventory_schema.ts
```

What it does:

- Creates `inventory` schema in `solo_ecommerce`
- Sets up `postgres_fdw` for cross-database access
- Creates all 12 tables with proper indexes and constraints
- Copies all data from `inventory_db.public.*` to `solo_ecommerce.inventory.*`
- Fixes sequences (setval to max ID)
- Creates views: `vw_products_complete`, `vw_current_inventory`
- Cleans up FDW resources

Step 2: Verify the Migration

```
npx ts-node scripts/verify_inventory_migration.ts
```

Verification checks:

- Row counts match between source and target
- Sample data matches (products, categories, brands)
- Views exist and are queryable
- Schema exists with all 12 tables

Step 3: Update Environment Variables

Edit `.env` file:

```
# Before
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/solo_ecommerce?schema=publ
INVENTORY_DATABASE_URL="postgresql://postgres:postgres@localhost:5432/inventory_db?sch

# After
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/solo_ecommerce?schema=publ
# INVENTORY_DATABASE_URL - no longer needed, can be removed
```

Step 4: Regenerate Prisma Client

```
npx prisma generate
```

This will generate a single Prisma client with access to both schemas.

Step 5: Restart the Backend

```
npm run start:dev
```

Step 6: Test API Endpoints

```
# Test products endpoint
curl http://localhost:3000/api/products

# Test categories endpoint
curl http://localhost:3000/api/categories

# Test brands endpoint
curl http://localhost:3000/api/brands
```

Rollback Procedure

If something goes wrong, you can rollback:

Option 1: Drop the inventory schema

```
-- Connect to solo_ecommerce
DROP SCHEMA IF EXISTS inventory CASCADE;
```

Then restore the original service files from git and restart.

Option 2: Restore from backup

```
# Drop and recreate databases
psql -U postgres -c "DROP DATABASE solo_ecommerce;"
psql -U postgres -c "CREATE DATABASE solo_ecommerce;"
psql -U postgres solo_ecommerce < solo_ecommerce_backup.sql

psql -U postgres -c "DROP DATABASE inventory_db;"
psql -U postgres -c "CREATE DATABASE inventory_db;"
psql -U postgres inventory_db < inventory_db_backup.sql
```

Schema Details

Prisma Model Name Mapping

Database Table	Prisma Model Name	Schema
inventory.countries	InvCountry	inventory
inventory.brands	InvBrand	inventory
inventory.designers	InvDesigner	inventory

Database Table	Prisma Model Name	Schema
inventory.categories	InvCategory	inventory
inventory.subcategories	InvSubcategory	inventory
inventory.products	InvProduct	inventory
inventory.product_dimensions	InvProductDimension	inventory
inventory.product_packaging	InvProductPackaging	inventory
inventory.product_pricing	InvProductPricing	inventory
inventory.product_images	InvProductImage	inventory
inventory.product_specifications	InvProductSpecification	inventory
inventory.inventory_transactions	InvInventoryTransaction	inventory

Service Changes

Service	Before	After
ProductsService	InventoryPrismaService	PrismaService
CategoriesService	InventoryPrismaService	PrismaService
BrandsService	InventoryPrismaService	PrismaService

Model Access Pattern

```
// Before
this.inventoryPrisma.product.findMany(...)

// After
this.prisma.invProduct.findMany(...)
```

Cleanup (After Successful Migration)

Once the migration is verified and the application is working:

1. Remove old files:

```
rm backend/prisma/schema-inventory.prisma
rm backend/src/prisma/inventory-prisma.service.ts
rm backend/src/prisma/inventory-prisma.module.ts
```

2. Remove `INVENTORY_DATABASE_URL` from `.env`
3. Drop the old database (optional, after extended testing):

```
DROP DATABASE inventory_db;
```

4. Clean up `package.json` scripts (remove inventory-specific scripts)

Troubleshooting

Error: "relation inventory.products does not exist"

Cause: Migration script hasn't been run or failed.

Solution: Run the migration script:

```
npx ts-node scripts/migrate_inventory_schema.ts
```

Error: "Cannot find module '@prisma/inventory-client'"

Cause: Old import statement still referencing the inventory client.

Solution:

1. Check for any remaining imports of `@prisma/inventory-client`
2. Update to use `@prisma/client` and the new `Inv*` model names

Error: "prisma.invProduct is undefined"

Cause: Prisma client hasn't been regenerated.

Solution:

```
npx prisma generate
```

Foreign Data Wrapper Errors

Cause: `postgres_fdw` not installed or user mapping failed.

Solution:

```
-- As superuser
CREATE EXTENSION IF NOT EXISTS postgres_fdw;

-- Grant usage
GRANT USAGE ON FOREIGN DATA WRAPPER postgres_fdw TO postgres;
```

npm Scripts Reference

Add these to `package.json`:

```
{
  "scripts": {
    "db:inventory:migrate": "ts-node scripts/migrate_inventory_schema.ts",
    "db:inventory:verify": "ts-node scripts/verify_inventory_migration.ts"
  }
}
```

Contact

For issues with this migration, check:

- GitHub Issues: <https://github.com/anydevice1234123-netizen/Updatedecember28>
- Backend logs: `npm run start:dev` output